ATLAS-ARC-001

Current Architecture and System Status

Project	ATLAS Master Flightline Clock
Document ID	ATLAS-ARC-001
Version	1.0
System Version	v1.6.0
Date	July 2026
Status	CURRENT AUTHORITATIVE

1. Purpose

This document defines the current implemented architecture and operational status of the ATLAS Master Flightline Clock system as of July 2026. It supersedes earlier conceptual architectural descriptions where they differ from the live codebase.

2. System Summary

ATLAS is a Python-based desktop timing and display application for F3K model glider competition operations. It provides an operator console and a public-facing display system driven by local system time and persisted schedule and settings data.

The system is intended to evolve toward a field-deployable appliance with minimal operator exposure to the host operating system.

3. Operational Responsibilities

ATLAS currently provides the following operational capabilities:

- Display of local real-time clock
- Management of scheduled heats
- Automatic activation of upcoming heats
- Timing of track-open and track-close windows
- Timing of remaining heat duration
- Display of heat number and thermalling direction
- Manual operator cancellation of active heats
- Persistent storage of settings and schedule data

4. Implemented Architecture

4.1 High-Level Structure

ATLAS follows a practical MVC-style architecture using PySide6 and SQLite.

Layer	Module	Responsibility
Application	`src/main.py`	Entry point, wiring, startup branching
Core	`src/core/engine.py`	Timing engine and state machine
Core	`src/core/database.py`	SQLite persistence and settings
Core	`src/core/models.py`	Data models (Pydantic)
Core (legacy)	`src/core/config.py`	Legacy JSON config — not active
Core (legacy)	`src/core/timers.py`	Legacy timer module — not active
GUI	`src/gui/windows.py`	Operator console (three-tab UI)
GUI	`src/gui/led_simulator.py`	LED-style software display backend
GUI	`src/gui/lcd_display.py`	Full-resolution monitor display backend
GUI	`src/gui/fonts.py`	Bitmap font definitions
GUI	`src/gui/themes.py`	UI theme and CSS styling

4.2 Data Flow

1. Application starts in `main.py`
2. Database is initialised
3. Engine is initialised with database access
4. Display backend is selected from settings
5. Operator console is optionally launched depending on auto-boot mode
6. Engine emits state updates via Qt signals at 1Hz
7. Display backend receives and renders the current model

4.3 Signal/Slot Model

The engine communicates with the display layer via Qt signals:

Signal	Connected To	Trigger

`model_updated`	`display.update_model(model)`	Every 1 second (1Hz tick)
`brightness_changed`	`display.set_brightness(value)`	Operator adjusts brightness slider

4.4 Display Backend System

ATLAS currently supports two display backends, selectable from the Settings tab:

Backend ID	Class	Description
`simulator`	`AtlasLEDSimulator`	Pixel-style discrete LED simulation. Supports 128x64 and 64x64 layouts.
`lcd`	`AtlasLCDDisplay`	Anti-aliased full-resolution display for standard monitors and TVs.

Backend selection is performed in `main.py` via a factory function based on the saved `display_backend` setting. Both backends implement identical public interfaces: `update_model()` and `set_brightness()`.

4.5 AtlasDisplayModel — Data Contract

The engine broadcasts state to the display via an `AtlasDisplayModel` Pydantic model:

Field	Type	Description
`local_time`	str	Current local time HH:MM:SS
`primary_timer`	str	Main countdown display MM:SS
`primary_timer_label`	str	Phase label text
`primary_timer_color`	str	Colour for timer display
`band_color`	str	Status band colour
`heat_number`	str	Current heat number
`thermalling_dir`	str	LEFT or RIGHT
`is_armed`	bool	Whether a heat is active
`planned_open`	str	Scheduled track open time
`planned_close`	str	Scheduled track close time

5. Persistence Model

5.1 Primary Persistence

ATLAS uses SQLite as the authoritative storage layer for heat schedule data and application settings. The database is located at `data/atlas.db` relative to the working directory at launch.

5.2 Database Tables

Table	Column	Type	Notes
heats	id	INTEGER PK	Auto-increment
heats	heat_number	TEXT	e.g. "01", "02"
heats	thermalling_dir	TEXT	LEFT or RIGHT
heats	track_open	INTEGER	Unix timestamp
heats	track_close	INTEGER	Unix timestamp
heats	heat_end	INTEGER	Unix timestamp
heats	status	TEXT	READY / ACTIVE / COMPLETED / CANCELLED
settings	key	TEXT PK	Setting name
settings	value	TEXT	All values stored as strings

5.3 Active Settings Keys

Key	Default	Description
`competition_name`	""	Event name string
`display_width`	"128"	Display width in pixels
`display_height`	"64"	Display height in pixels
`led_scale`	"8"	LED pixel scale factor
`brightness`	"80"	Brightness 10–100
`display_backend`	"simulator"	"simulator" or "lcd"
`auto_boot_enabled`	"0"	"1" = display-only boot, "0" = normal

5.4 Legacy Configuration

> `atlas_config.json` and `src/core/config.py` remain in the repository but are not the live configuration source for the running application.

6. Runtime Behaviour

6.1 Startup Modes

Mode	Condition	Behaviour
Normal	`auto_boot_enabled = "0"`	Launches operator console and public display
Auto-Boot	`auto_boot_enabled = "1"`	Launches public display only — appliance mode

6.2 Heat Lifecycle

The engine runs on a 1-second timing cycle and follows this race lifecycle:

State	Display Colour	Label
No heats scheduled	Grey	WAITING
Next heat > 15 min away	Grey	WAITING
Next heat ≤ 15 min — auto-activates	Yellow	TRACK OPENS IN
Track open — pilots may launch	Green	ENTER NOW
Track closed — heat still running	Red	HEAT TIME REMAINING
Heat ended	Grey	COMPLETED

6.3 Operator Functions

Tab	Function
LIVE	Live status display, abort/cancel active heat
SCHEDULE	Add, edit, delete heats; view full schedule
SETTINGS	Competition name, display backend, display size, LED scale, brightness, auto-boot, restart, exit

7. Technical Debt and Known Issues

Item	Risk	Action
Database path uses relative `data/atlas.db`	Medium — path resolves relative to launch directory	Harden to project-relative absolute path
`src/core/config.py` — legacy	Low	Document as legacy or formally retire
`src/core/timers.py` — legacy	Low	Document as legacy or formally retire
UI wording "EXIT TO MACOS"	Low — cosmetic	Generalise for cross-platform deployment
Startup scripts are macOS-specific	Medium	Create Windows equivalents

8. Current Truth Precedence

Where previous documentation conflicts with this document or with the current codebase, the following precedence applies:

1. Current source code
2. Current deployment-validated runtime behaviour
3. This architecture and status document (ATLAS-ARC-001)
4. July 2026 handover dossier
5. Older architecture, review, and status documents

9. Immediate Priorities

- Refresh formal documentation set — in progress
- Harden database path handling for cross-platform reliability
- Prepare and validate Windows deployment process
- Validate appliance-style startup on target deployment hardware
- Continue toward physical display hardware integration